



AERO-GCS Flight Endurance Estimator

Version 1.0

Technical Documentation & System Manual

An internal flight-operations tool for estimating drone flight endurance under real-world atmospheric conditions, with telemetry-based model validation and coefficient tuning.

Author: Aadarsh Sinha & Krishnashish Das

Organization: Wingspan Global

Document Revision: 1.0

Strictly Confidential

This document is a proprietary specification of Wingspan Global. All implementations must conform to the structures described herein. Unauthorized distribution is prohibited.

Document Control

Field	Value
Document Title	AERO-GCS Flight Endurance Estimator — Technical Documentation
Document ID	WG-FEE-DOC-001
Version	1.0
Status	Released
Author	Aadarsh Sinha & Krishnashish Das
Organization	Wingspann Global
Release Date	July 18, 2026
Last Updated	July 18, 2026
Distribution	Internal Use Only — Flight Operations Team
Classification	Strictly Confidential

Critical

Document Security Notice: This document contains proprietary information belonging to Wingspann Global. It is intended exclusively for authorized personnel involved in drone flight operations. Do not copy, distribute, or share this document without explicit written authorization from the engineering team lead.

Revision History

Version	Date	Author(s)	Changes
1.0	July 18, 2026	Aadarsh Sinha, Krishnashish Das	Initial document creation. Full system architecture, physics model derivation, UI walkthrough, and deployment guide.

Contents

Document Control	1
Revision History	2
Abstract	5
1 Introduction	6
1.1 Problem Statement	6
1.2 Design Goals	6
2 System Architecture	7
2.1 Technology Stack	7
2.2 System Architecture Diagram	7
2.3 Directory Structure	8
3 Database Schema	9
3.1 Aircraft Table	9
3.2 FlightLog Table	9
3.3 WeatherCache Table	9
4 Physics Model — Flight Endurance Calculation	10
4.1 Stage 1: Payload Current Scaling	10
4.2 Stage 2: Nameplate Hover Time	10
4.3 Stage 3: Usable Capacity Limit	10
4.4 Stage 4: LiPo Temperature Derating	10
4.5 Stage 5: Wind Current Penalty	11
4.6 Stage 6: Density Altitude Penalty	11
4.7 Range Estimation	11
4.8 Safety Advisories	11
5 Weather Integration	12
5.1 Open-Meteo API	12
5.2 Offline Caching Strategy	12
6 User Interface	13
6.1 Dashboard Overview	13
6.2 HUD Flight Timer	13
6.3 Derating Waterfall Diagnostics	14
6.4 Model Tuning & Coefficient Regression	15
6.5 Flight Logs & Validation Table	16
7 API Reference	17
8 Testing & Validation	18
8.1 Unit Tests — Physics Engine	18
8.2 Integration Tests — API Endpoints	18

9	Deployment & Usage	19
9.1	Prerequisites	19
9.2	Installation	19
9.3	Running the Application	19
9.4	Typical Workflow	19
10	Security Considerations	20
10.1	Operational Constraints	20
A	Appendix A: Glossary	21
B	Appendix B: References	21

Abstract

The **AERO-GCS Flight Endurance Estimator** (FEE) is an internal web-based tool developed for the Wingspann Global flight-operations team. It computes the estimated hover endurance and cruise range of multi-rotor UAV platforms under real-world conditions, accounting for battery chemistry, payload mass, ambient temperature, wind loading, and density altitude.

Key features include:

- **Physics-Based Estimation:** Piecewise LiPo temperature derating, wind-drag current scaling, barometric density-altitude penalties, and momentum-theory payload adjustment.
- **Live Weather Sync:** Integration with the Open-Meteo API (no API key required) for real-time temperature, wind speed, and site elevation, with automatic offline caching.
- **Telemetry Validation:** Flight-log recording against saved aircraft profiles, with predicted-vs-actual scatter regression and tunable model coefficients.
- **GCS-Themed Interface:** Dark navy HUD-style dashboard with JetBrains Mono telemetry readouts, a horizontal derating diagnostics waterfall, and inline safety advisories.
- **Data Export:** Session and flight-log CSV export, plus browser-based PDF report generation.

1 Introduction

1.1 Problem Statement

Flight operations teams routinely need to estimate how long a specific drone airframe can remain aloft under a given set of conditions. Off-the-shelf calculators typically apply only a single “efficiency factor” and ignore critical environmental variables such as ambient temperature (which affects LiPo internal resistance), wind speed (which increases motor current draw to maintain station), and site altitude (which reduces propeller lift efficiency due to lower air density).

The Flight Endurance Estimator was designed to bridge this gap by providing a multi-variable, physics-grounded model that can be *tuned against real telemetry* rather than trusted blindly.

1.2 Design Goals

1. **Accurate, Multi-Variable Physics:** Go beyond simple capacity-over-current calculations by incorporating temperature, wind, altitude, and payload derating curves derived from published LiPo datasheets and aerodynamic theory.
2. **Operator-Friendly Interface:** Present results in a GCS-familiar aesthetic (dark background, monospace telemetry fonts, inline advisory banners) so that flight crews can interpret outputs at a glance.
3. **Model Calibration:** Allow operators to log actual flight times and adjust model sensitivity coefficients per-aircraft, building confidence in predictions over time.
4. **Offline Resilience:** Cache weather readings locally so the tool remains functional in field environments with limited connectivity.
5. **Auditability:** Export session data and flight logs as CSV/PDF for inclusion in flight-operations records.

Note

The estimator is designed as a **planning aid**, not a safety-of-flight instrument. Operators must always apply their own judgment and adhere to organizational SOPs regarding minimum reserve levels and weather limits.

2 System Architecture

2.1 Technology Stack

The application is built on a two-tier architecture:

- **Backend:** Python 3.14, FastAPI, SQLAlchemy (ORM), SQLite (persistence), HTTPX (async HTTP client for weather API).
- **Frontend:** Vanilla HTML5, CSS3, and ES6 JavaScript — no external UI frameworks or charting libraries. Custom SVG rendering for all charts.
- **Deployment:** Uvicorn ASGI server, single `run.bat` bootstrap launcher.

2.2 System Architecture Diagram

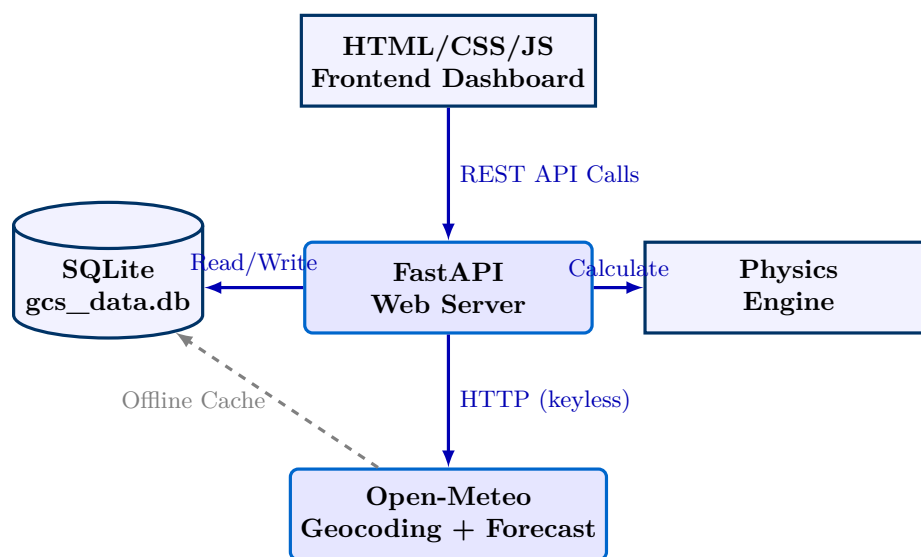


Figure 1: AERO-GCS Flight Endurance Estimator — System Architecture

2.3 Directory Structure

Path	Description
backend/main.py	FastAPI application entry point. Defines all REST API endpoints, CORS middleware, and mounts static frontend files.
backend/database.py	SQLAlchemy engine, session factory, and SQLite foreign-key enforcement.
backend/models.py	ORM models: <code>Aircraft</code> , <code>FlightLog</code> , <code>WeatherCache</code> .
backend/schemas.py	Pydantic request/response validation schemas.
backend/physics.py	Core derating calculations (temperature, wind, altitude, payload).
backend/weather.py	Open-Meteo API integration with geocoding and offline caching.
backend/test_physics.py	Automated unit tests for all physics calculations.
backend/test_api.py	Automated API integration tests.
frontend/index.html	Single-page application structure.
frontend/css/style.css	AERO-GCS dark navy theme, responsive layout, print stylesheet.
frontend/js/api.js	REST client wrapper (fetch-based).
frontend/js/app.js	UI state management, form handlers, real-time calculation mirror.
frontend/js/charts.js	Custom SVG waterfall and scatter-plot renderers.
run.bat	Windows bootstrap launcher (activates venv, starts Uvicorn on port 7006).
gcs_data.db	SQLite database file (auto-created on first run).

Table 2: Project File Structure

3 Database Schema

The application uses SQLite via SQLAlchemy. Three tables persist all operational data.

3.1 Aircraft Table

Stores saved drone configuration profiles. Each profile includes physical specifications, battery parameters, and per-aircraft tuning coefficients.

Column	Type	Description
id	INTEGER PK	Auto-incrementing identifier
name	TEXT UNIQUE	Human-readable aircraft identifier
empty_weight	REAL	Airframe weight without payload (kg)
motors	INTEGER	Number of motors
hover_current	REAL	Measured hover current per motor at empty weight (A)
prop_size	TEXT	Propeller dimensions (e.g., "15x5.2")
battery_capacity	INTEGER	Battery capacity (mAh)
battery_cells	INTEGER	Cell count (S)
battery_reserve	REAL	Land-with reserve percentage
battery_efficiency	REAL	Discharge efficiency percentage
cruise_speed	REAL	Optional cruise speed (km/h)
temp_coeff	REAL	Temperature sensitivity coefficient (default 1.0)
wind_coeff	REAL	Wind sensitivity coefficient (default 1.0)
alt_coeff	REAL	Altitude sensitivity coefficient (default 1.0)
created_at	DATETIME	Profile creation timestamp

Table 3: Aircraft Profile Table Schema

3.2 FlightLog Table

Records of actual flights with measured conditions and duration, linked to an aircraft profile via foreign key.

3.3 WeatherCache Table

Stores the last-fetched weather reading per location (keyed by place name or coordinate pair). Used for offline fallback when the Open-Meteo API is unreachable.

4 Physics Model — Flight Endurance Calculation

The estimator applies a cascading series of derating factors to the theoretical nameplate hover time. Each stage is displayed in the UI as a step in the “derating waterfall” diagnostics panel.

4.1 Stage 1: Payload Current Scaling

Using momentum theory, hover current scales non-linearly with total weight:

$$I_{\text{hover}} = I_{\text{hover, empty}} \times \left(\frac{W_{\text{empty}} + W_{\text{payload}}}{W_{\text{empty}}} \right)^{1.3} \quad (1)$$

The exponent of 1.3 is derived from the relationship between thrust, power, and rotor disc loading in momentum theory. If no payload is specified, the scale factor defaults to 1.0.

4.2 Stage 2: Nameplate Hover Time

The theoretical maximum hover time at the loaded current draw:

$$t_{\text{nameplate}} = \frac{C_{\text{mAh}}/1000}{N_{\text{motors}} \times I_{\text{hover}}} \times 60 \quad [\text{minutes}] \quad (2)$$

4.3 Stage 3: Usable Capacity Limit

Accounting for the land-with reserve and discharge efficiency losses:

$$t_{\text{usable}} = t_{\text{nameplate}} \times \left(1 - \frac{R_{\text{reserve}}}{100} \right) \times \frac{\eta_{\text{efficiency}}}{100} \quad (3)$$

4.4 Stage 4: LiPo Temperature Derating

A piecewise function models the capacity reduction due to battery temperature:

$$d_{\text{temp}}(T) = \begin{cases} 0.50 & T \leq -20^{\circ}\text{C} \\ 0.50 + 0.50 \times \frac{T + 20}{40} & -20^{\circ}\text{C} < T < 20^{\circ}\text{C} \\ 1.00 & 20^{\circ}\text{C} \leq T \leq 35^{\circ}\text{C} \\ 1.00 - 0.15 \times \frac{T - 35}{20} & 35^{\circ}\text{C} < T \leq 55^{\circ}\text{C} \\ 0.85 & T > 55^{\circ}\text{C} \end{cases} \quad (4)$$

Applied with the tunable sensitivity coefficient C_{temp} :

$$d_{\text{temp, adj}} = 1.0 - (1.0 - d_{\text{temp}}) \times C_{\text{temp}} \quad (5)$$

$$t_{\text{temp}} = t_{\text{usable}} \times d_{\text{temp, adj}} \quad (6)$$

Warning

At temperatures below 0°C, LiPo cells suffer significant internal resistance increases and voltage sag. The estimator will surface an inline advisory recommending battery pre-heating before takeoff.

4.5 Stage 5: Wind Current Penalty

Motor current draw increases to maintain station in wind. The penalty is modeled as a piecewise linear function:

$$p_{\text{wind}}(W) = \begin{cases} 1.00 & W \leq 10 \text{ km/h} \\ 1.00 + 0.65 \times \frac{W - 10}{40} & 10 < W \leq 50 \text{ km/h} \\ 1.65 & W > 50 \text{ km/h} \end{cases} \quad (7)$$

Applied with tuning coefficient:

$$p_{\text{wind, adj}} = 1.0 + (p_{\text{wind}} - 1.0) \times C_{\text{wind}} \quad (8)$$

$$t_{\text{wind}} = t_{\text{temp}} / p_{\text{wind, adj}} \quad (9)$$

4.6 Stage 6: Density Altitude Penalty

At higher altitudes, air density decreases, requiring higher motor RPM (and thus more current) to produce the same thrust:

$$\rho_{\text{ratio}} = \exp\left(-\frac{\text{altitude}}{8500}\right) \quad (10)$$

$$p_{\text{alt}} = \frac{1}{\sqrt{\rho_{\text{ratio}}}} = \exp\left(\frac{\text{altitude}}{17000}\right) \quad (11)$$

$$t_{\text{final}} = t_{\text{wind}} / p_{\text{alt, adj}} \quad (12)$$

4.7 Range Estimation

If a cruise speed v_{cruise} is provided:

$$R_{\text{range}} = \frac{t_{\text{final}}}{60} \times v_{\text{cruise}} \quad [\text{km}] \quad (13)$$

4.8 Safety Advisories

The system surfaces non-blocking inline advisories (not modal dialogs) when:

- Wind speed exceeds 25 km/h (**warning**) or 40 km/h (**critical**)
- Temperature is below 0°C or above 45°C (**warning**)
- Site altitude exceeds 3000 m (**warning**)
- Estimated flight time is under 3 minutes (**critical**)

5 Weather Integration

5.1 Open-Meteo API

The system uses the [Open-Meteo](#) forecast and geocoding APIs, which require no API key. This eliminates credential management overhead for field deployments.

Geocoding API

Resolves place names (e.g., “San Francisco, CA”) to latitude/longitude coordinates. Returns the official place name, administrative region, and elevation.

Forecast API

Returns current conditions for given coordinates: `temperature_2m`, `wind_speed_10m`, `relative_humidity_2m`, and site `elevation`.

5.2 Offline Caching Strategy

Every successful weather fetch is persisted to the `WeatherCache` table under three keys:

1. `last_fetched` — always overwritten with the most recent query.
2. `coord:{lat},{lon}` — keyed by rounded coordinates for coordinate-based lookups.
3. `name:{query}` — keyed by the search term for name-based lookups.

If the API is unreachable, the system attempts cache recovery in order: specific coordinate/-name cache first, then the global `last_fetched` entry. If no cache exists, a clear error message instructs the operator to enter conditions manually.

6 User Interface

The frontend follows the AERO-GCS ground control station visual language: dark navy backgrounds, cyan accent colors, JetBrains Mono for telemetry data, Inter for labels, and an industrial, panel-based layout.

6.1 Dashboard Overview



Figure 2: AERO-GCS Flight Endurance Estimator — Main Dashboard View. The left column contains the aircraft profile configuration and environmental controls. The right column displays the HUD timer, advisories, waterfall diagnostics, and model tuning panels.

6.2 HUD Flight Timer

The primary output is a large, monospace digital readout displaying the estimated flight endurance in MM:SS format. This readout updates in real-time as sliders and inputs are adjusted. Secondary telemetry values (takeoff mass, cruise range, loaded hover current) are displayed

alongside.

6.3 Derating Waterfall Diagnostics

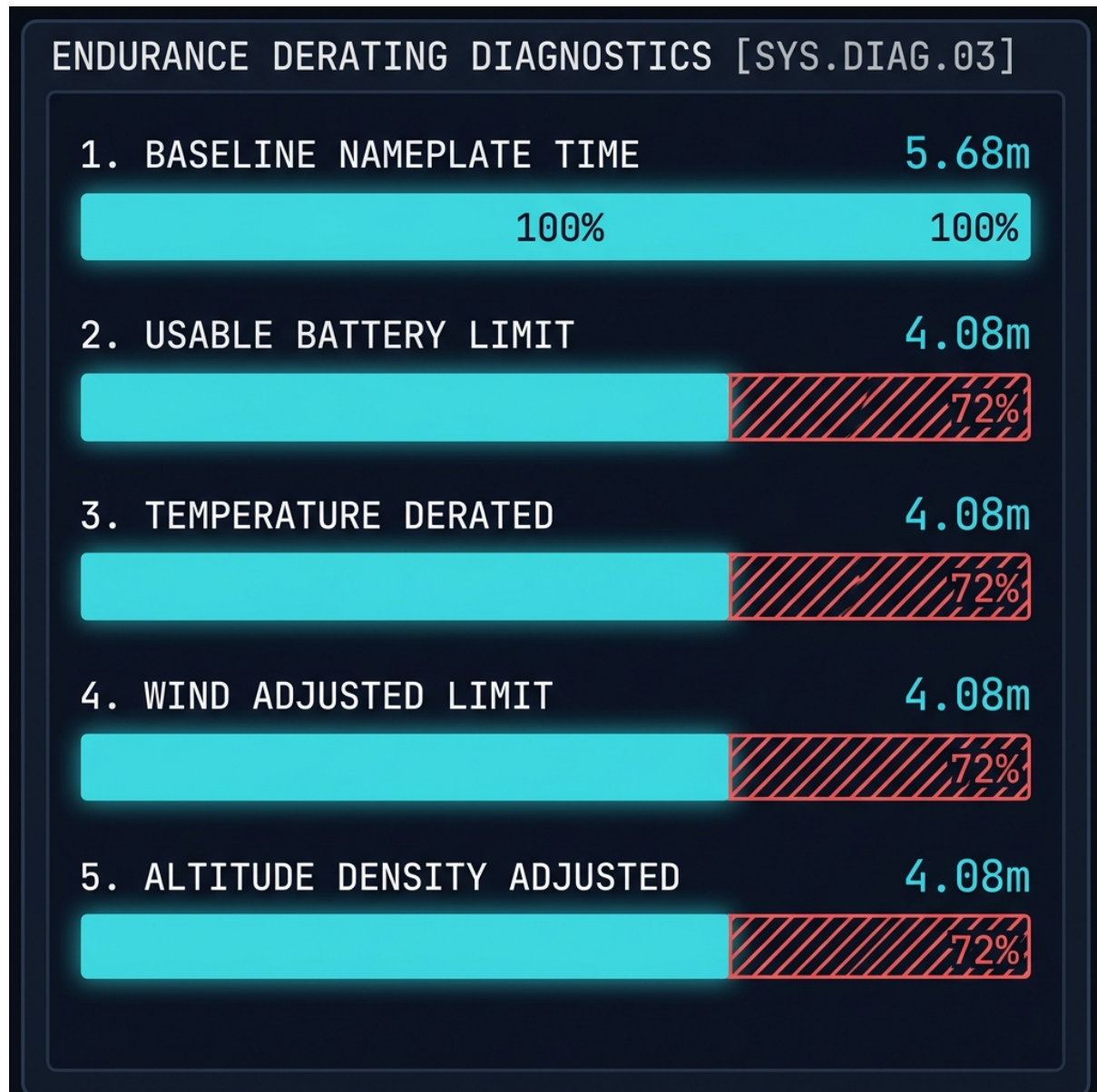


Figure 3: Endurance Derating Diagnostics — Horizontal step-bar waterfall showing progressive capacity reduction through each derating stage. Cyan bars represent remaining capacity; hatched sections indicate losses.

The waterfall reads like a GCS diagnostics panel rather than a generic chart. Each row shows:

- The stage label (e.g., “2. USABLE BATTERY LIMIT”)
- The resulting flight time and percentage of nameplate remaining
- A horizontal bar proportional to the nameplate time, with the loss portion rendered as a hatched overlay

6.4 Model Tuning & Coefficient Regression

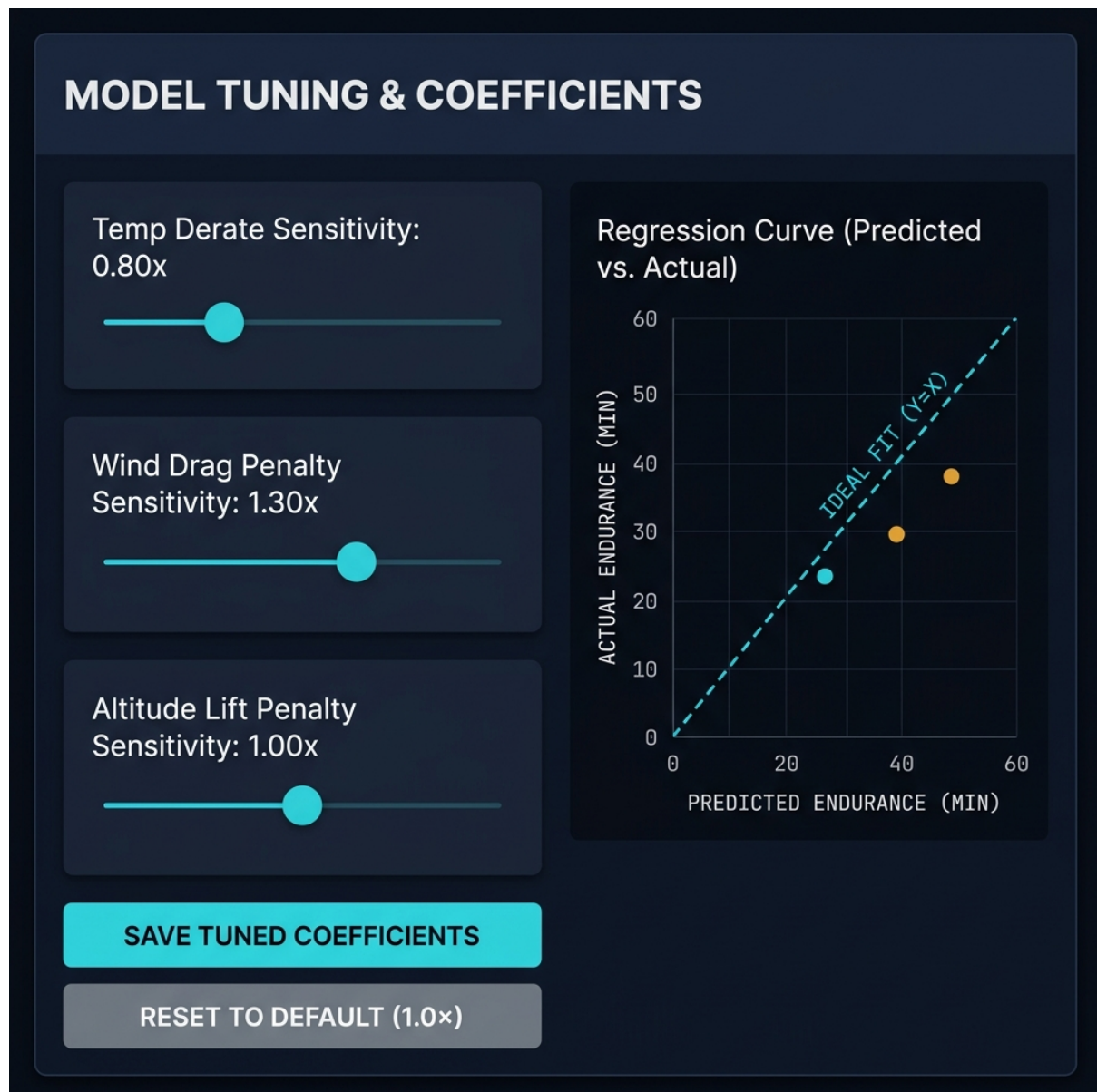


Figure 4: Model Tuning Panel — Sensitivity coefficient sliders (left) and predicted-vs-actual scatter regression plot (right). Dots are color-coded by prediction accuracy: cyan (<15% error), amber (15–30%), red (>30%).

The tuning panel allows operators to adjust three coefficients per aircraft:

- **Temperature Derate Sensitivity** (C_{temp}): Amplifies or reduces the LiPo temperature penalty.
- **Wind Drag Penalty Sensitivity** (C_{wind}): Amplifies or reduces the wind current scaling.
- **Altitude Lift Penalty Sensitivity** (C_{alt}): Amplifies or reduces the density altitude effect.

As sliders are dragged, the scatter plot updates at 60 fps using a client-side JavaScript mirror of the backend physics calculations. This allows operators to visually align prediction dots with the ideal $y = x$ regression line before saving coefficients to the aircraft profile.

6.5 Flight Logs & Validation Table

The “Flight Op Logs & Validation” tab provides:

- A log submission form that auto-captures current environmental slider values as flight conditions.
- A scrollable table showing all logged flights with columns: Date, Payload, Environment (T/W/A), Actual time, Predicted time, and prediction error percentage.
- CSV export of the full log history for a selected aircraft.

7 API Reference

All endpoints are served by FastAPI on `http://localhost:7006`. Interactive Swagger documentation is available at `/docs`.

Method	Endpoint	Description
POST	<code>/api/aircraft</code>	Create a new aircraft profile.
GET	<code>/api/aircraft</code>	List all saved aircraft profiles.
GET	<code>/api/aircraft/{id}</code>	Retrieve a specific aircraft profile.
PUT	<code>/api/aircraft/{id}</code>	Update aircraft specifications.
PUT	<code>/api/aircraft/{id}/coefficients</code>	Update tuning coefficients only.
DELETE	<code>/api/aircraft/{id}</code>	Delete profile and all linked logs.
POST	<code>/api/flight-logs</code>	Record a new flight log entry.
GET	<code>/api/flight-logs/{aircraft_id}</code>	List logs for a specific aircraft.
DELETE	<code>/api/flight-logs/{log_id}</code>	Delete a flight log entry.
GET	<code>/api/weather?q={name}</code>	Fetch weather by place name.
GET	<code>/api/weather?lat={lat}&lon={lon}</code>	Fetch weather by coordinates.
POST	<code>/api/estimate</code>	Run endurance calculation.
GET	<code>/api/export/csv?aircraft_id={id}</code>	Export flight logs as CSV.
GET	<code>/api/export/session-csv?...</code>	Export session inputs + result as CSV.

Table 4: REST API Endpoint Summary

8 Testing & Validation

8.1 Unit Tests — Physics Engine

The file `backend/test_physics.py` contains automated assertions for all piecewise curves:

```

1 =====
2  RUNNING FLIGHT ENDURANCE ESTIMATOR PHYSICS TESTS
3  =====
4  Testing LiPo Temperature Derating Curve...
5  [PASS] LiPo Temperature curves verified.
6  Testing Wind Drag Penalty Curve...
7  [PASS] Wind Drag penalties verified.
8  Testing Density Altitude Penalty Curve...
9  [PASS] Density Altitude scaling verified.
10 Testing Full Estimation Model Calculations...
11 [PASS] Full estimation model math verified.
12 Testing Tuning Coefficient Effects...
13 [PASS] Sensitivity coefficient overrides verified.
14 =====
15          ALL TESTS COMPLETED SUCCESS
16 =====

```

Listing 1: Physics test execution output

8.2 Integration Tests — API Endpoints

The file `backend/test_api.py` performs end-to-end validation:

```

1 =====
2  RUNNING INTEGRATION TESTS FOR API ON PORT 7006
3  =====
4  [INFO] Creating aircraft profile...
5  [PASS] Profile saved successfully with ID: 2
6  [INFO] Querying /api/estimate endpoint...
7  [INFO] API Estimation Results:
8    - Final Estimated Time: 4.08 mins
9    - Loaded Hover Current: 14.53 A/motor
10   - Cruise Range: 4.08 km
11   - Number of Advisories: 0
12 [INFO] Cleaning up test profile...
13 =====
14          SUCCESS: API INTEGRATION TESTS PASSED!
15 =====

```

Listing 2: API integration test execution output

9 Deployment & Usage

9.1 Prerequisites

- Python 3.10 or later
- Internet connection for initial dependency installation and weather sync (optional after first use)

9.2 Installation

1. Clone or copy the project directory to the target machine.
2. Open a terminal in the project root and run:

```
1 python -m venv .venv
2 .venv\Scripts\pip install -r backend\requirements.txt
```

Listing 3: Virtual environment setup

9.3 Running the Application

Double-click `run.bat` or execute it from a terminal. The console will display:

```
1 =====
2 AERO-GCS FLIGHT ENDURANCE ESTIMATOR BOOTSTRAP
3 Developed by Aadarsh Sinha & Krishnashish Das
4 =====
5
6 [INFO] Starting FastAPI App server...
7 [INFO] Service will be available at http://localhost:7006/
```

Listing 4: Application startup output

The browser will automatically open to `http://localhost:7006/`.

9.4 Typical Workflow

1. **Create/Select Aircraft:** Enter airframe specs and save, or select a previously saved profile from the dropdown.
2. **Set Environment:** Use the weather sync to pull live conditions, or manually adjust the temperature, wind, altitude, and humidity sliders.
3. **Read Results:** The HUD timer, waterfall, and advisories update in real-time.
4. **Log Flights:** After a real flight, record the actual duration in the Flight Logs tab. The system auto-captures the current environmental slider values.
5. **Tune Model:** Switch to the Model Tuning tab and adjust coefficients until predicted times align with logged actuals on the scatter plot. Save coefficients to the profile.
6. **Export:** Download session CSV or print a PDF report for flight-ops records.

10 Security Considerations

Critical

The AERO-GCS Flight Endurance Estimator is designed for internal use on trusted networks. It does not implement authentication or authorization.

10.1 Operational Constraints

- **Network Binding:** The server binds to 127.0.0.1 by default, restricting access to the local machine only.
- **No External Data Ingestion:** The only outbound connections are to Open-Meteo (a public, keyless API). No user data is transmitted externally.
- **SQLite File Security:** The `gcs_data.db` file contains aircraft configurations and flight logs. Protect it according to your organization's data classification policy.
- **Input Validation:** All API inputs are validated by Pydantic schemas with strict range constraints (e.g., temperature -50°C to 70°C , wind 0–150 km/h).

A Appendix A: Glossary

AUW (All-Up Weight)

Total weight of the aircraft including airframe, battery, and payload.

Cell Count (S)

Number of LiPo cells in series. A 6S battery has 6 cells at 3.7V nominal each (22.2V total).

Density Altitude

Altitude corrected for non-standard atmospheric conditions. Higher density altitude means thinner air.

Discharge Efficiency

The percentage of nominal battery capacity that is practically available. Accounts for voltage sag, cell imbalance, and connector losses.

FEE

Flight Endurance Estimator.

GCS

Ground Control Station.

HUD

Heads-Up Display.

Land-With Reserve

The percentage of battery capacity reserved for safe return-to-home and landing.

MSL

Mean Sea Level — the reference datum for altitude measurements.

Momentum Theory

Aerodynamic model relating rotor thrust, disc area, and induced velocity to predict hover power requirements.

Nameplate Time

Theoretical maximum hover time using 100% of battery capacity at the calculated hover current, with no environmental derating.

B Appendix B: References

- Open-Meteo API Documentation: <https://open-meteo.com/en/docs>
- FastAPI Official Documentation: <https://fastapi.tiangolo.com/>
- LiPo Battery Temperature Performance — various manufacturer datasheets (Tattu, Turnigy, GenStatu)
- Johnson, W. (2013). *Rotorcraft Aeromechanics*. Cambridge University Press.
- Standard Atmosphere Model: $\rho = \rho_0 \exp(-h/H)$, with scale height $H \approx 8500$ m